ChordDetectr:

Real Time Audio Note and Chord Detection System

EC327 Project Documentation
Spring 2026

Paul Nguyen, Wilfred Iheukwu, Ryan Shi

1 Introduction

This project implements a real-time audio signal processing system capable of detecting musical notes and inferring chords from live microphone input. The system processes raw audio data and converts it into structured musical information through a combination of digital signal processing techniques, frequency-domain analysis, and heuristic classification. The result is a visualization tool that displays waveform data, frequency spectra, detected notes, and predicted chords with an associated confidence metric.

2 Project Overview and Goals

The primary objective of this project is to design and implement a complete real-time audio processing pipeline that transforms continuous audio input into discrete musical interpretations. This involves capturing audio, performing spectral analysis, identifying the strongest frequencies, mapping those frequencies to musical notes, and ultimately inferring chords based on detected pitch classes. A secondary objective is to create a responsive and interpretable graphical interface that allows users to observe both the raw signal and the processed results simultaneously. The system is designed to operate under real-world conditions, meaning it must handle noise, signal instability, and varying input quality without significant degradation in performance.

At a higher level, the project demonstrates the integration of multiple domains, including digital signal processing, data structures, real-time systems, and user interface design. It is meant to work as a cohesive system rather than a collection of isolated components.

3 System Operation and Flow

The system begins by capturing audio using a streaming input interface. Incoming samples are stored in a fixed-length buffer, ensuring that only the most recent data is analyzed. Once enough data is available, the system applies a Hanning window to reduce spectral leakage and computes the Fast Fourier Transform (FFT) to obtain the signal's frequency spectrum. From this spectrum, the system identifies local peaks corresponding to dominant frequencies. These peaks are filtered to remove low-magnitude noise and frequencies outside the musically relevant range. The remaining frequencies are converted into MIDI note values and reduced to pitch classes, which represent notes independent of octave.

Chord detection is performed by comparing the detected pitch classes against a predefined chord library. Each chord candidate is evaluated using a scoring function that balances overlap, extra notes, and missing notes. The best-scoring chord is selected if it meets a minimum threshold. To mitigate instability, the system applies temporal smoothing by aggregating recent detections and selecting the most consistent results. The final output is displayed through a graphical interface, which updates in real time.

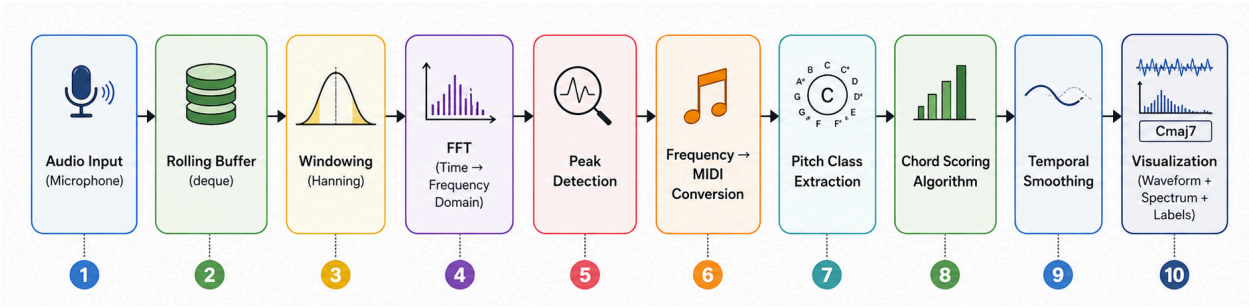


Figure 1. Logical Flow of ChordDetectr Program

3.1 Audio Input

The system begins by capturing live audio input using a streaming callback. Incoming samples are continuously appended to a rolling buffer, ensuring that only the most recent data is retained for processing.

```
def audiocb(indata, frames, time, status):
    mono = np.squeeze(indata).astype(np.float32)
    audiobuf.extend(mono)
```

This buffer forms the basis for all subsequent analysis. Once enough samples are collected, the system applies a Hanning window and computes the Fast Fourier Transform to obtain the frequency spectrum.

```
window = np.hanning(len(x))
xw = x * window
spectrum = np.fft.rfft(xw)
freqs = np.fft.rfftfreq(len(xw), d=1.0 / SAMPLERATE)
mags = np.abs(spectrum)
```

The resulting spectrum is then filtered and analyzed to identify dominant frequency components. Peak detection is performed by scanning for local maxima in the magnitude array.

```
def findpeaks(mags):
    peaks = []
    for i in range(1, len(mags) - 1):
        if mags[i] > mags[i - 1] and mags[i] > mags[i + 1]:
            peaks.append(i)
    return np.array(peaks)
```

After identifying candidate frequencies, the system converts them into musical representations. Frequencies are mapped to MIDI note values using a logarithmic relationship, then reduced to pitch classes.

```
def freqmidi(freq):
```

```
    return int(round(69 + 12 * np.log2(freq / 440.0)))

def freqpc(freq):
    return freqmidi(freq) % 12
```

These pitch classes are then used as input to the chord detection algorithm.

3.2 Audio Processing & Chord Detection

Chord detection is implemented using a predefined library of pitch class sets. Each chord is represented as a set of integers corresponding to its constituent notes.

```
CHORDLIB = {
    "C major": {0, 4, 7},
    "A minor": {9, 0, 4},
    ...
}
```

The system evaluates each possible chord using a scoring function that balances correct matches, extra notes, and missing notes.

```
overlap = len(detected & chordtones)
extra = len(detected - chordtones)
missing = len(chordtones - detected)

score = 3.0 * overlap - 1.2 * extra - 0.8 * missing
```

The chord with the highest score is selected, provided it exceeds a minimum threshold. A confidence value is also computed to indicate reliability.

```
confidence = bestoverlap / (bestoverlap + bestextra +
bestmissing + 1e-6)
```

3.3 Temporal Smoothing

To reduce instability, the system aggregates recent pitch detections and filters out inconsistent results. Only pitch classes that appear multiple times within a short history window are retained.

```
def smoothpc(pitchclasses):
    recentpitches.append(set(pitchclasses))
    counts = {}
    for s in recentpitches:
        for pc in s:
            counts[pc] = counts.get(pc, 0) + 1

    stable = [pc for pc, count in counts.items() if count >= 2]
    return sorted(stable)
```

Chord predictions are similarly stabilized by tracking recent outputs and selecting the most frequent result over time.

3.4 Visualization

The graphical interface updates in real time using a timer-driven loop. Both waveform and spectrum plots are refreshed at a fixed interval.

```
self.timer = QtCore.QTimer()
self.timer.timeout.connect(self.updateplots)
self.timer.start(UPDATEMS)
```

Within each update cycle, the system processes audio data, computes detections, and updates the display:

```
self.wavecurve.setData(x)
self.speccurve.setData(fplot, mplot)

self.infolabel.setText(
   f"Detected notes: {self.shownnotes} | "
   f"Chord guess: {self.shownchord} | "
   f"Confidence: {self.shownconf:.2f}"
)
```

This tight integration between processing and visualization enables immediate feedback and reinforces the connection between raw signal data and interpreted musical output.

4 Example Behavior

When a user plays a simple triad, such as a C major chord, the system identifies the dominant frequencies corresponding to C, E, and G. These are mapped to their respective pitch classes, and the chord detection algorithm correctly classifies the input as “C major” with a high confidence value. In cases where the input signal is noisy or incomplete, the system may detect only a subset of the expected notes, resulting in a lower confidence score or an “Unknown” classification.

For live instruments such as a guitar or piano, the system may initially produce unstable results due to transient noise and harmonics. However, within a short time window, the smoothing mechanisms stabilize the output, yielding consistent note and chord predictions. As this is a live program, it would require an audio input which we are unable to display in words. Please refer to our video to see a real example of how the program functions in real time.

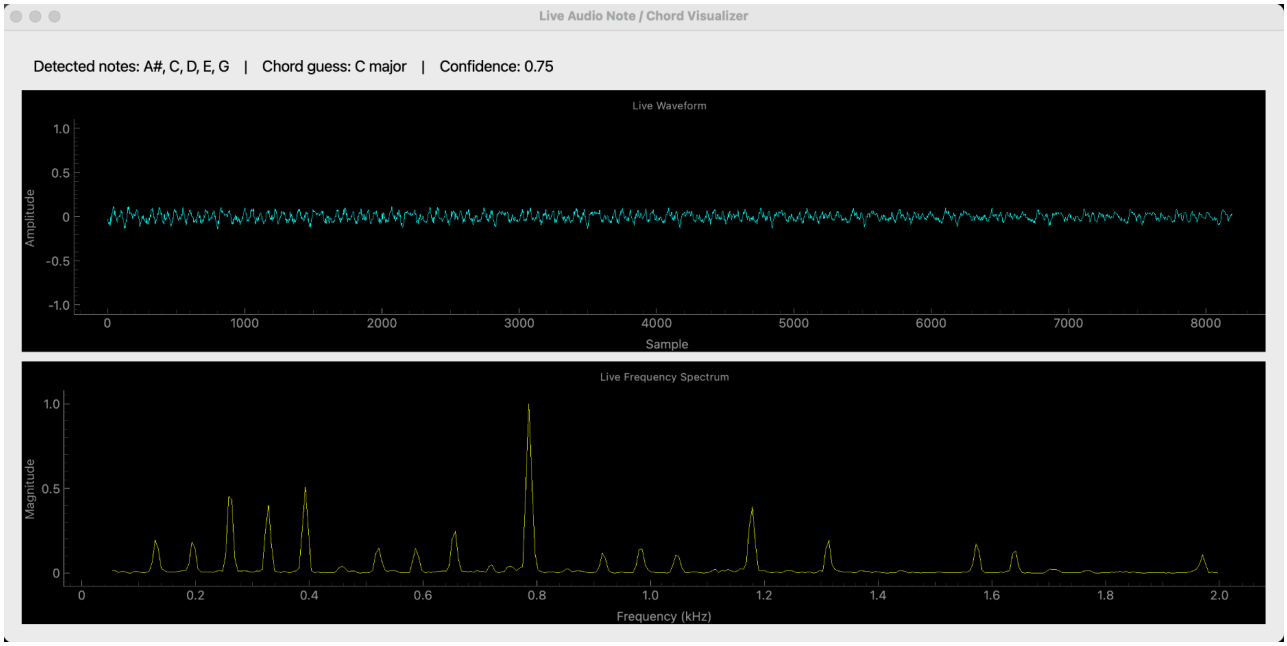


Figure 2. Example Pilot GUI of C Major Chord on Guitar Detected

The graphical interface was refined using AI-assisted iteration to improve visual clarity and overall usability. The initial layouts were functional but lacked consistency in spacing, color balance, and visual hierarchy. By using AI to suggest interface improvements such as simplified color schemes, cleaner typography, and more structured layout organization, the design was made more cohesive. The final interface prioritizes readability and smooth visual flow while maintaining real-time performance, resulting in a display that is both intuitive and visually polished.



Figure 3. Final Rendition of ChordDetectr GUI using AI

5 Design Choices and Tradeoffs

Several key design decisions were made to balance performance, accuracy, and interpretability. The use of FFT-based spectral analysis was chosen due to its computational efficiency and suitability for real-time applications. However, this approach introduces limitations in frequency resolution and can be sensitive to harmonic overtones, which may be incorrectly interpreted as distinct notes.

Peak-based frequency detection was selected for its simplicity and speed. While more advanced methods such as harmonic product spectrum or machine learning models could improve accuracy, they would significantly increase computational complexity and reduce transparency. The current approach prioritizes interpretability and responsiveness over absolute precision.

The system restricts analysis to a frequency range of 50 Hz to 2000 Hz to eliminate irrelevant noise and focus on the most musically significant region. This improves robustness but limits the system's ability to detect very low or very high pitches.

A heuristic chord scoring model was implemented instead of a trained classification model. This decision ensures that the system remains lightweight and explainable, allowing users to understand how chord predictions are made. The tradeoff is that the model may be less accurate in complex or ambiguous musical contexts.

Temporal smoothing was introduced to address instability in real-time detection. While this significantly improves output consistency, it introduces a small delay between input and displayed results. This latency is an intentional compromise to achieve a more stable user experience.

8 Justification and Significance

This project satisfies the criteria of a full-scale class project due to its scope, integration of multiple technical domains, and level of system complexity. Unlike a typical homework assignment, which might focus on a single concept such as FFT computation or basic signal analysis, this project requires the design and implementation of an end-to-end system. It involves real-time data processing, synchronization between computational and graphical components, and the handling of imperfect real-world input. Additionally, the project requires careful consideration of algorithmic tradeoffs, system performance, and user interaction, all of which are characteristic of larger engineering efforts.

The project is notable for its ability to transform raw audio signals into structured musical information in real time. This capability is inherently engaging, as it allows users to observe the relationship between sound, frequency, and musical structure as it happens. From a technical perspective, the project demonstrates how abstract mathematical concepts such as Fourier transforms and logarithmic frequency scaling can be applied to a practical and interactive application. The integration of signal processing, music theory, and visualization results in a system that is both analytically meaningful and intuitively compelling.